

Blocks

Set Wheel Speed

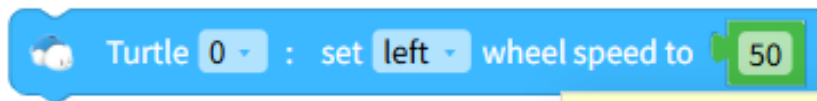
Sets the Turtle's wheel speed.

If the wheel speed is positive, it rotates forward, and if the wheel speed is negative, it rotates backward.

For example, if the wheel speed is 100, it rotates forward at speed 100,

and if the wheel speed is -100, it rotates backward at speed 100.

Once you set the wheel speed, the Turtle will keep moving at that speed until you set the wheel speed again.



Set the wheel speed. The wheel speed ranges from -100 to 100.

Dropdown Options and Input Values

Name	Type	Description	Range / Type
wheel	Dropdown Option	Wheel type	left, right, both
speed	Input Value	Wheel speed	Integer -100 ~ 100, 0: stop

JavaScript Code

```
// Set left wheel speed to 50
if($('Turtle*0:wheel.move').d != 0) {
    $('Turtle*0:wheel.move').d = 0;
}
$('Turtle*0:wheel.speed.left').d = __getSpeed('Turtle*0', 50);

// Set right wheel speed to 40
if($('Turtle*0:wheel.move').d != 0) {
    $('Turtle*0:wheel.move').d = 0;
}
$('Turtle*0:wheel.speed.right').d = __getSpeed('Turtle*0', 40);
```

```
// Set both wheel speeds to 30
if($('Turtle*0:wheel.move').d != 0) {
    $('Turtle*0:wheel.move').d = 0;
}
$('Turtle*0:wheel.speed.left').d = __getSpeed('Turtle*0', 30);
$('Turtle*0:wheel.speed.right').d = __getSpeed('Turtle*0', 30);
```

Python Code

```
# Set left wheel speed to 50
if __('Turtle*0:wheel.move').d != 0:
    __('Turtle*0:wheel.move').d = 0
__('Turtle*0:wheel.speed.left').d = __getSpeed('Turtle*0', 50)

# Set right wheel speed to 40
if __('Turtle*0:wheel.move').d != 0:
    __('Turtle*0:wheel.move').d = 0
__('Turtle*0:wheel.speed.right').d = __getSpeed('Turtle*0', 40)

# Set both wheel speeds to 30
if __('Turtle*0:wheel.move').d != 0:
    __('Turtle*0:wheel.move').d = 0
__('Turtle*0:wheel.speed.left').d = __getSpeed('Turtle*0', 30)
__('Turtle*0:wheel.speed.right').d = __getSpeed('Turtle*0', 30)
```

Move by Distance

Sets the distance the Turtle will move.

If wheel speed is not set, it will not move.

If the distance value is 0, it will keep moving without stopping at the currently set wheel speed.

If you check “wait”, it waits until the move is completed.

If you check “wait”, it can only be used inside an async function.



✓ inch
cm
mm

Set the distance for Turtle to move.
If the wheel speed is not set, it won't move.
If the distance is 0, it will continue moving according to the wheel speed.
If wait is checked, it will wait until move completes.

Dropdown Options and Input Values

Name	Type	Description	Range / Type
distance	Input Value	Distance value	Real number $\neq 0$
unit	Dropdown Option	Distance unit	cm, mm, inch

JavaScript Code

```
// Move 50cm | Wait 0
$('Turtle*0:wheel.move').d = __getDistance('Turtle*0', 50, 'cm');
await $('Turtle*0:wheel.!move').w();

// Move 50mm | Wait X
$('Turtle*0:wheel.move').d = __getDistance('Turtle*0', 50, 'mm');

// Move 50inch | Wait X
$('Turtle*0:wheel.move').d = __getDistance('Turtle*0', 50, 'inch');
```

Python Code

```
# Move 50cm | Wait 0
__('Turtle*0:wheel.move').d = __getDistance('Turtle*0', 50, 'cm')
await __('Turtle*0:wheel.!move').w()

# Move 50mm | Wait X
__('Turtle*0:wheel.move').d = __getDistance('Turtle*0', 50, 'mm')
```

```
# Move 50inch | Wait X
__('Turtle*0:wheel.move').d = __getDistance('Turtle*0', 50, 'inch')
```

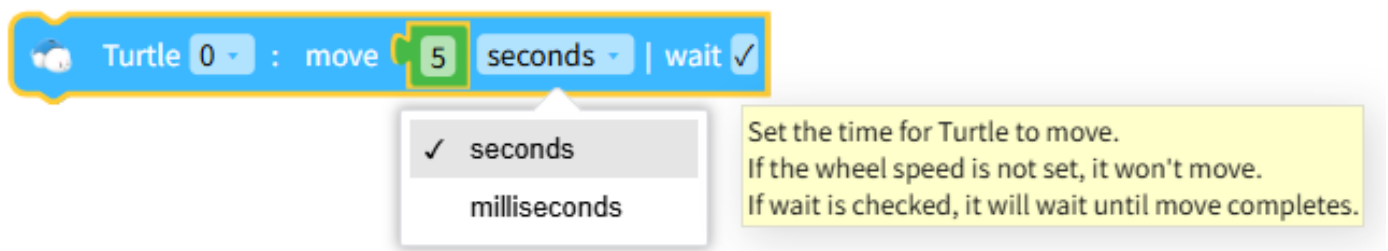
Move by Time

Sets the time the Turtle will move.

If wheel speed is not set, it will not move.

If you check “wait”, it waits until the move is completed.

If you check “wait”, it can only be used inside an async function.



Dropdown Options and Input Values

Name	Type	Description	Range / Type
time	Input Value	Time value	Real number ≥ 0
unit	Dropdown Option	Time unit	seconds, milliseconds

If you set the option to milliseconds, the input will be the value of time divided by 1000.

JavaScript Code

```
// Move 5 seconds | Wait 0
await __stopAfterDelay('Turtle*0', 5, true);

// Move 5 milliseconds | Wait X
__stopAfterDelay('Turtle*0', 0.005, false);
```

Python Code

```
# Move 5 seconds | Wait 0
await __stopAfterDelay('Turtle*0', 5, True)

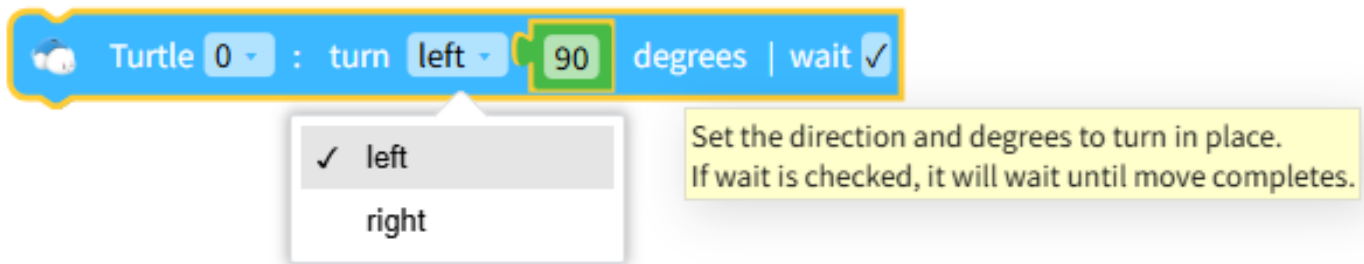
# Move 5 milliseconds | Wait X
__stopAfterDelay('Turtle*0', 0.005, False)
```

Turn in Place

Sets the direction and degree for the Turtle to rotate in place.

If you check “wait”, it waits until the move is completed.

If you check “wait”, it can only be used inside an async function.



Dropdown Options and Input Values

Name	Type	Description	Range / Type
direction	Dropdown Option	Turn direction	left, right
degree	Input Value	Turn degree	Integer ≥ 0

JavaScript Code

```
// Turn left 90 degrees in place | Wait 0
await __turn_degree_left('Turtle*0', 90, true);

// Turn right 270 degrees in place | Wait X
__turn_degree_right('Turtle*0', 270, false);
```

Python Code

```
# Turn left 90 degrees in place | Wait 0
await __turn_degree_left('Turtle*0', 90, True)

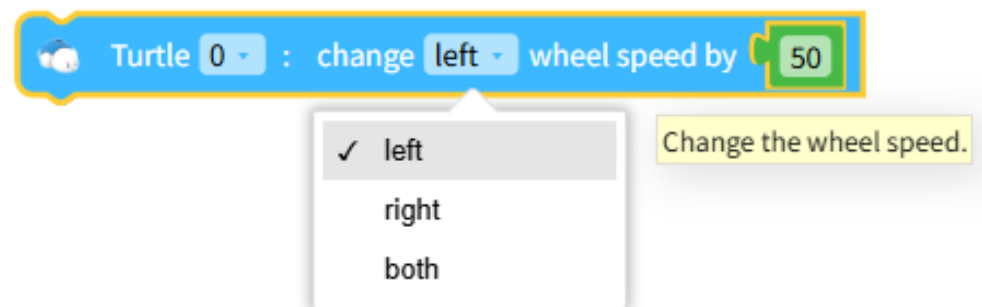
# Turn right 270 degrees in place | Wait X
__turn_degree_right('Turtle*0', 270, False)
```

Change Wheel Speed

Changes the Turtle's wheel speed.

The new wheel speed becomes the current wheel speed plus the input value.

The new wheel speed is clamped to the range -100 ~ 100.



Dropdown Options and Input Values

Name	Type	Description	Range / Type
wheel	Dropdown Option	Wheel type	left, right, left & right
speed	Input Value	Value to add to current speed	Integer -200 ~ 200

JavaScript Code

```
// Change left wheel speed by +50
if($('Turtle*0:wheel.move').d != 0) {
    $('Turtle*0:wheel.move').d = 0;
}

$('Turtle*0:wheel.speed.left').d = $('Turtle*0:wheel.speed.left').d +
__getSpeed('Turtle*0', 50);
```

```

// Change right wheel speed by -40
if($('Turtle*0:wheel.move').d != 0) {
    $('Turtle*0:wheel.move').d = 0;
}
$('Turtle*0:wheel.speed.right').d = $('Turtle*0:wheel.speed.right').d +
__getSpeed('Turtle*0', (-40));

// Change both wheel speeds by +200
if($('Turtle*0:wheel.move').d != 0) {
    $('Turtle*0:wheel.move').d = 0;
}
$('Turtle*0:wheel.speed.left').d = $('Turtle*0:wheel.speed.left').d +
__getSpeed('Turtle*0', 200);
$('Turtle*0:wheel.speed.right').d = $('Turtle*0:wheel.speed.right').d +
__getSpeed('Turtle*0', 200);

```

Python Code

```

# Change left wheel speed by +50
if __('Turtle*0:wheel.move').d != 0:
    __('Turtle*0:wheel.move').d = 0
__('Turtle*0:wheel.speed.left').d = __('Turtle*0:wheel.speed.left').d +
__getSpeed('Turtle*0', 50)

# Change right wheel speed by -40
if __('Turtle*0:wheel.move').d != 0:
    __('Turtle*0:wheel.move').d = 0
__('Turtle*0:wheel.speed.right').d = __('Turtle*0:wheel.speed.right').d +
__getSpeed('Turtle*0', (-40))

# Change both wheel speeds by +200
if __('Turtle*0:wheel.move').d != 0:
    __('Turtle*0:wheel.move').d = 0

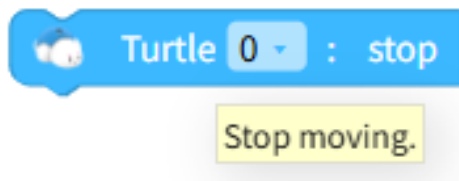
```

```
__('Turtle*0:wheel.speed.left').d = __('Turtle*0:wheel.speed.left').d +
__getSpeed('Turtle*0', 200)
__('Turtle*0:wheel.speed.right').d = __('Turtle*0:wheel.speed.right').d +
__getSpeed('Turtle*0', 200)
```

Stop

Stops the Turtle's movement.

Both wheel speeds are reset to 0.



JavaScript Code

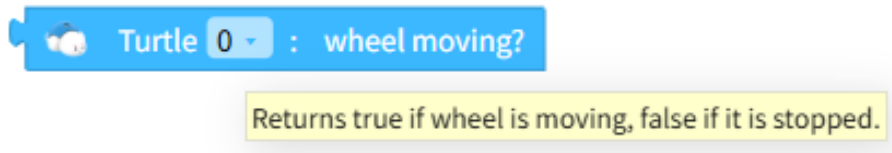
```
__stopMove('Turtle*0');
```

Python Code

```
__stopMove('Turtle*0')
```

Are the Wheels Moving?

Returns whether the Turtle's wheels are moving as **True (1) / False (0)**.



JavaScript Code

```
$('Turtle*0:wheel.speed.left').d !== 0 || $('Turtle*0:wheel.speed.right').d !== 0
```


Python Code

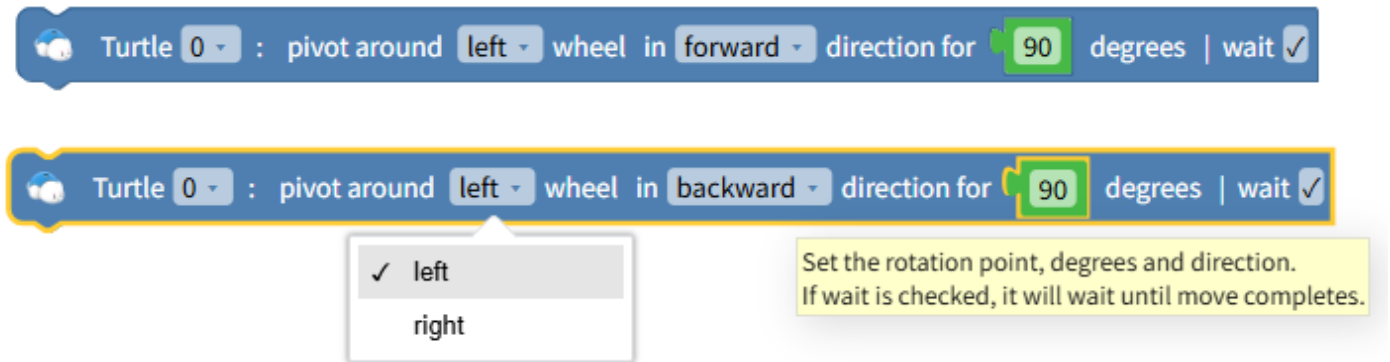
```
__('Turtle*0:wheel.speed.left').d != 0 or __('Turtle*0:wheel.speed.right').d != 0
```

Pivot Turn (Rotate Around a Wheel)

Sets the pivot wheel, direction, and angle for the Turtle to rotate in place.

If you check “wait”, it waits until the move is completed.

If you check “wait”, it can only be used inside an async function.



Dropdown Options and Input Values

Name	Type	Description	Range / Type
pivot	Dropdown Option	Pivot wheel	left wheel, right wheel
direction	Dropdown Option	Direction	forward, backward
degree	Input Value	Rotation angle	Integer ≥ 0

JavaScript Code

```
// Pivot on left wheel, forward, 90 degrees | Wait 0
await __pivot('Turtle*0', 'left wheel', 'forward', 90, true);

// Pivot on left wheel, backward, 90 degrees | Wait X
__pivot('Turtle*0', 'left wheel', 'backward', 90, false);

// Pivot on right wheel, forward, 90 degrees | Wait 0
await __pivot('Turtle*0', 'right wheel', 'forward', 90, true);
```

```
// Pivot on right wheel, backward, 90 degrees | Wait X
__pivot('Turtle*0', 'right wheel', 'backward', 90, false);
```

Python Code

```
# Pivot on left wheel, forward, 90 degrees | Wait 0
await __pivot('Turtle*0', 'left wheel', 'forward', 90, True)

# Pivot on left wheel, backward, 90 degrees | Wait X
__pivot('Turtle*0', 'left wheel', 'backward', 90, False)

# Pivot on right wheel, forward, 90 degrees | Wait 0
await __pivot('Turtle*0', 'right wheel', 'forward', 90, True)

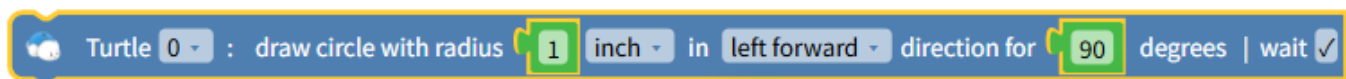
# Pivot on right wheel, backward, 90 degrees | Wait X
__pivot('Turtle*0', 'right wheel', 'backward', 90, False)
```

Draw a Circle While Turning

Sets the direction, radius, and angle when drawing a circle using the pen.

If you check “wait”, it waits until the move is completed.

If you check “wait”, it can only be used inside an async function.



Set degrees, radius and direction when drawing circle using a pen.
If wait is checked, it will wait until move completes.

Dropdown Options and Input Values

Name	Type	Description	Range / Type
direction	Dropdown Option	Direction	left forward, left backward, right forward, right backward
radius	Input Value	Circle radius	Real number ≥ 0
unit	Dropdown Option	Distance unit	cm, mm, inch
degree	Input Value	Rotation angle	Real number ≥ 0

JavaScript Code

```
// Turn left forward, draw a circle with radius 1cm, rotate 90 degrees | Wait 0
await __pivot_circle('Turtle*0', '', 'left forward', __getDistance('Turtle*0', 1,
'cm'), 90, true); //(robot, pivot, direction, radius, degree, wait_w)

// Turn left backward, draw a circle with radius 1mm, rotate 90 degrees | Wait 0
await __pivot_circle('Turtle*0', '', 'left backward', __getDistance('Turtle*0',
1, 'mm'), 90, true); //(robot, pivot, direction, radius, degree, wait_w)

// Turn right forward, draw a circle with radius 1inch, rotate 90 degrees | Wait X
__pivot_circle('Turtle*0', '', 'right forward', __getDistance('Turtle*0', 1,
'inch'), 90, false); //(robot, pivot, direction, radius, degree, wait_w)

// Turn right backward, draw a circle with radius 1cm, rotate 90 degrees | Wait X
__pivot_circle('Turtle*0', '', 'right backward', __getDistance('Turtle*0', 1,
'cm'), 90, false); //(robot, pivot, direction, radius, degree, wait_w)
```

Python Code

```
# Turn left forward, draw a circle with radius 1cm, rotate 90 degrees | Wait 0
await __pivot_circle('Turtle*0', '', 'left forward', __getDistance('Turtle*0',
1, 'cm'), 90, True) #(robot, pivot, direction, radius, degree, wait_w)
```

```
# Turn left backward, draw a circle with radius 1mm, rotate 90 degrees | Wait 0
await __pivot_circle('Turtle*0', '', 'left backward', __getDistance('Turtle*0',
1, 'mm'), 90, True) #(robot, pivot, direction, radius, degree, wait_w)

# Turn right forward, draw a circle with radius 1inch, rotate 90 degrees | Wait X
__pivot_circle('Turtle*0', '', 'right forward', __getDistance('Turtle*0', 1,
'inch'), 90, False) #(robot, pivot, direction, radius, degree, wait_w)

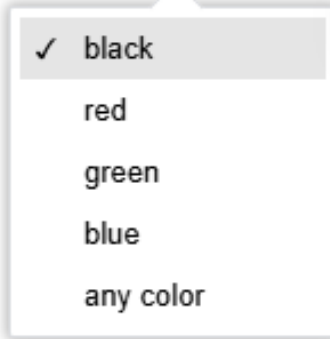
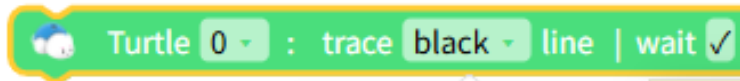
# Turn right backward, draw a circle with radius 1cm, rotate 90 degrees | Wait X
__pivot_circle('Turtle*0', '', 'right backward', __getDistance('Turtle*0', 1,
'cm'), 90, False) #(robot, pivot, direction, radius, degree, wait_w)
```

Follow a Line Using Sensor

The Turtle follows a specific line using the bottom color sensor.

If you check “wait”, it waits until the move is completed.

If you check “wait”, it can only be used inside an async function.



Using the color sensor, move along a line of specific color.

Dropdown Options and Input Values

Name	Type	Description	Range / Type
color	Dropdown Option	Line color	black(8), red(9), green(11), blue(13), any(15)

JavaScript Code

```
// Follow a black line (8) | Wait 0
$('Turtle*0:trace.mode').d = 8;
await $('Turtle*0:trace.!mode').w();

// Follow a red line (9) | Wait 0
$('Turtle*0:trace.mode').d = 9;
await $('Turtle*0:trace.!mode').w();

// Follow a green line (11) | Wait 0
$('Turtle*0:trace.mode').d = 11;
await $('Turtle*0:trace.!mode').w();

// Follow a blue line (13) | Wait X
$('Turtle*0:trace.mode').d = 13;

// Follow any line (15) | Wait X
$('Turtle*0:trace.mode').d = 15;
```

Python Code

```
# Follow a black line (8) | Wait 0
__('Turtle*0:trace.mode').d = 8
await __('Turtle*0:trace.!mode').w()

# Follow a red line (9) | Wait 0
__('Turtle*0:trace.mode').d = 9
```

```

await __('Turtle*0:trace.!mode').w()

# Follow a green line (11) | Wait 0
__('Turtle*0:trace.mode').d = 11
await __('Turtle*0:trace.!mode').w()

# Follow a blue line (13) | Wait X
__('Turtle*0:trace.mode').d = 13

# Follow any line (15) | Wait X
__('Turtle*0:trace.mode').d = 15

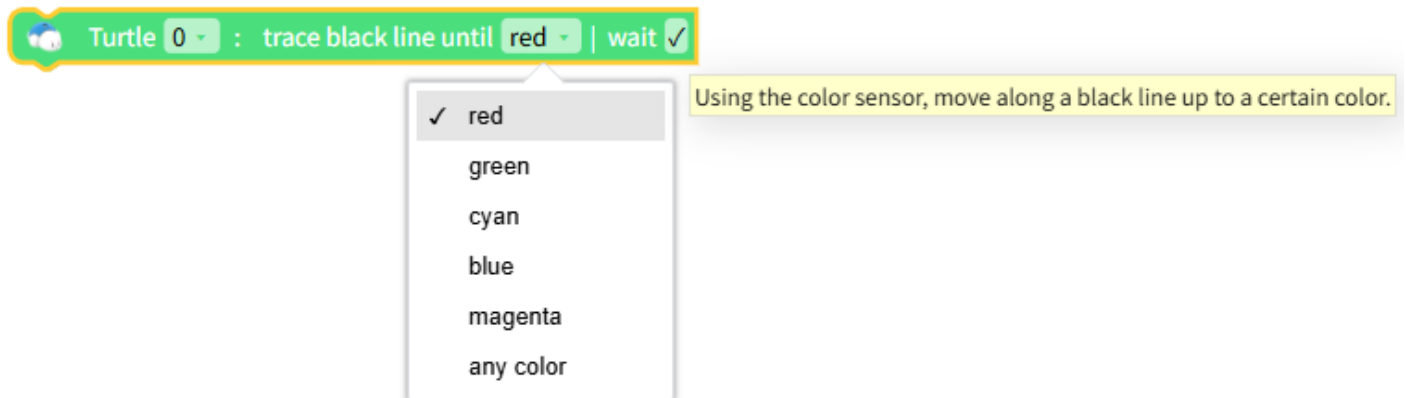
```

Follow Black Line Until a Specific Color

Using the bottom color sensor, the Turtle follows a black line until it encounters the specified color.

If you check “wait”, it waits until the move is completed.

If you check “wait”, it can only be used inside an async function.



Dropdown Options and Input Values

Name	Type	Description	Range / Type
color	Dropdown Option	Target color	red(49), green(51), cyan(52), blue(53), magenta(54), any(55)

JavaScript Code

```
// Follow black line until red (49) | Wait 0
$('Turtle*0:trace.mode').d = 49;
await $('Turtle*0:trace.!mode').w();

// Follow black line until green (51) | Wait 0
$('Turtle*0:trace.mode').d = 51;
await $('Turtle*0:trace.!mode').w();

// Follow black line until cyan (52) | Wait 0
$('Turtle*0:trace.mode').d = 52;
await $('Turtle*0:trace.!mode').w();

// Follow black line until blue (53) | Wait X
$('Turtle*0:trace.mode').d = 53;

// Follow black line until magenta (54) | Wait X
$('Turtle*0:trace.mode').d = 54;

// Follow black line until any color (55) | Wait X
$('Turtle*0:trace.mode').d = 55;
```

Python Code

```
# Follow black line until red (49) | Wait 0
__('Turtle*0:trace.mode').d = 49
await __('Turtle*0:trace.!mode').w()

# Follow black line until green (51) | Wait 0
__('Turtle*0:trace.mode').d = 51
await __('Turtle*0:trace.!mode').w()

# Follow black line until cyan (52) | Wait 0
```

```

__('Turtle*0:trace.mode').d = 52
await __('Turtle*0:trace.!mode').w()

# Follow black line until blue (53) | Wait X
__('Turtle*0:trace.mode').d = 53

# Follow black line until magenta (54) | Wait X
__('Turtle*0:trace.mode').d = 54

# Follow black line until any color (55) | Wait X
__('Turtle*0:trace.mode').d = 55

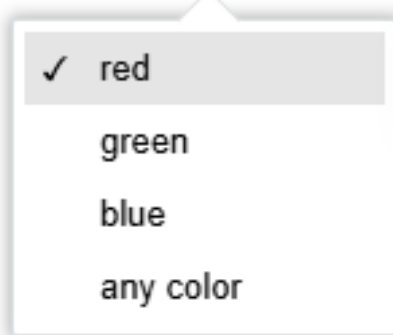
```

Follow a Specific Color Line Until Black

Using the bottom color sensor, the Turtle follows a specific color line until it encounters black.

If you check “wait”, it waits until the move is completed.

If you check “wait”, it can only be used inside an async function.



Using the color sensor, move along a line of specific color up to black color.

Dropdown Options and Input Values

Name	Type	Description	Range / Type
color	Dropdown Option	Line color	red(57), green(59), blue(61), any(63)

JavaScript Code

```
// Follow red line until black (57) | Wait 0
$('Turtle*0:trace.mode').d = 57;
await $('Turtle*0:trace.!mode').w();

// Follow green line until black (59) | Wait 0
$('Turtle*0:trace.mode').d = 59;
await $('Turtle*0:trace.!mode').w();

// Follow blue line until black (61) | Wait X
$('Turtle*0:trace.mode').d = 61;

// Follow any color line until black (63) | Wait X
$('Turtle*0:trace.mode').d = 63;
```

Python Code

```
# Follow red line until black (57) | Wait 0
__('Turtle*0:trace.mode').d = 57
await __('Turtle*0:trace.!mode').w()

# Follow green line until black (59) | Wait 0
__('Turtle*0:trace.mode').d = 59
await __('Turtle*0:trace.!mode').w()

# Follow blue line until black (61) | Wait X
__('Turtle*0:trace.mode').d = 61

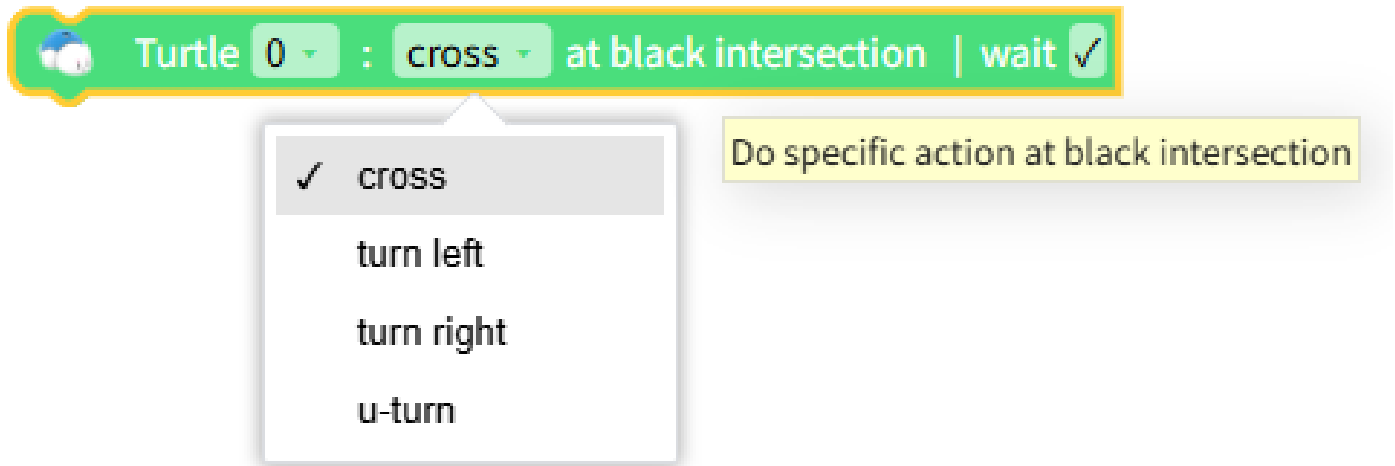
# Follow any color line until black (63) | Wait X
__('Turtle*0:trace.mode').d = 63
```

Behavior at a Black Intersection

Using the bottom color sensor, when the Turtle detects a black intersection it performs the specified behavior.

If you check “wait”, it waits until the move is completed.

If you check “wait”, it can only be used inside an async function.



Dropdown Options and Input Values

Name	Type	Description	Range / Type
behavior	Dropdown Option	Action at black intersection	move forward(16), turn left(24), turn right(32), u-turn(40)

JavaScript Code

```
// At black intersection: move forward (16) | Wait 0
$('Turtle*0:trace.mode').d = 16;
await $('Turtle*0:trace.!mode').w();

// At black intersection: turn left (24) | Wait 0
$('Turtle*0:trace.mode').d = 24;
await $('Turtle*0:trace.!mode').w();

// At black intersection: turn right (32) | Wait X
```

```
$('Turtle*0:trace.mode').d = 32;  
  
// At black intersection: u-turn (40) | Wait X  
$('Turtle*0:trace.mode').d = 40;
```

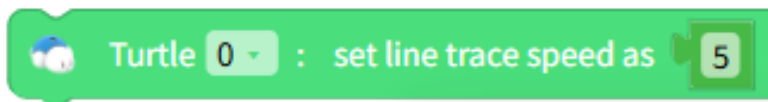
Python Code

```
# At black intersection: move forward (16) | Wait 0  
__('Turtle*0:trace.mode').d = 16  
await __('Turtle*0:trace.!mode').w()  
  
# At black intersection: turn left (24) | Wait 0  
__('Turtle*0:trace.mode').d = 24  
await __('Turtle*0:trace.!mode').w()  
  
# At black intersection: turn right (32) | Wait X  
__('Turtle*0:trace.mode').d = 32  
  
# At black intersection: u-turn (40) | Wait X  
__('Turtle*0:trace.mode').d = 40
```

Set Line Tracing Speed

Sets the Turtle's line tracing speed.

Speed range is 1 ~ 10.



Set the speed of Turtle's tracing the line.
The line trace speed ranges from 1 to 10.

Dropdown Options and Input Values

Name	Type	Description	Range / Type
speed	Input Value	line tracing speed	Integer 1 ~ 10

JavaScript Code

```
// Set line tracing speed to 5
$('Turtle*0:trace.speed').d = 5;
```

Python Code

```
# Set line tracing speed to 5
__('Turtle*0:trace.speed').d = 5
```

Set Line Tracing Gain

Sets the direction change amount (gain) for line tracing.

Gain range is 1 ~ 10.



Set the amount of change in line tracing direction.
The line trace gain ranges from 1 to 10.

Dropdown Options and Input Values

Name	Type	Description	Range / Type
gain	Input Value	line tracing gain	Integer 1 ~ 10

JavaScript Code

```
__('Turtle*0:trace.gain').d = 5; // Set direction gain to 5
```

Python Code

```
__('Turtle*0:trace.gain').d = 5 # Set direction gain to 5
```

Stop Line Tracing

Stops the Turtle's line tracing feature.



Terminate the line tracing function.

JavaScript Code

```
$('Turtle*0:trace.mode').d = 0; // Stop line tracing
```

Python Code

```
__('Turtle*0:trace.mode').d = 0 # Stop line tracing
```

Set LED Color

Sets the Turtle's head LED color.



Set the head LED color.

- ✓ black
- red
- yellow
- green
- cyan
- blue
- magenta
- white

Dropdown Options and Input Values

Name	Type	Description	Range / Type
color	Dropdown Option	Color	black([0, 0, 0]), red([255, 0, 0]), yellow([255, 255, 0]), green([0, 255, 0]) cyan([0, 255, 255]), blue([0, 0, 255]), magenta([255, 0, 255]), white([255, 255, 255])

JavaScript Code

```
// Set head LED color to black
$('Turtle*0:head_led').d = [0, 0, 0];

// Set head LED color to red
$('Turtle*0:head_led').d = [255, 0, 0];

// Set head LED color to yellow
$('Turtle*0:head_led').d = [255, 255, 0];

// Set head LED color to green
$('Turtle*0:head_led').d = [0, 255, 0];

// Set head LED color to cyan
$('Turtle*0:head_led').d = [0, 255, 255];

// Set head LED color to blue
$('Turtle*0:head_led').d = [0, 0, 255];

// Set head LED color to magenta
$('Turtle*0:head_led').d = [255, 0, 255];
```

```
// Set head LED color to white
$('Turtle*0:head_led').d = [255, 255, 255];
```

Python Code

```
# Set head LED color to black
__('Turtle*0:head_led').d = [0, 0, 0]

# Set head LED color to red
__('Turtle*0:head_led').d = [255, 0, 0]

# Set head LED color to yellow
__('Turtle*0:head_led').d = [255, 255, 0]

# Set head LED color to green
__('Turtle*0:head_led').d = [0, 255, 0]

# Set head LED color to cyan
__('Turtle*0:head_led').d = [0, 255, 255]

# Set head LED color to blue
__('Turtle*0:head_led').d = [0, 0, 255]

# Set head LED color to magenta
__('Turtle*0:head_led').d = [255, 0, 255]

# Set head LED color to white
__('Turtle*0:head_led').d = [255, 255, 255]
```

Set LED Color Using Color Category Blocks

Sets the Turtle's head LED color using blocks in the color category.



Dropdown Options and Input Values

Name	Type	Description	Range / Type
color	Input Value	LED color	RGB array

JavaScript Code

```
// Set head LED color to R:255, G:0, B:0
$('Turtle*0:head_led').d = [255, 0, 0];

// Set head LED color to R:255, G:255, B:0
$('Turtle*0:head_led').d = [255, 255, 0];

// Set head LED color to random color
$('Turtle*0:head_led').d = __randomColor();
```

Python Code

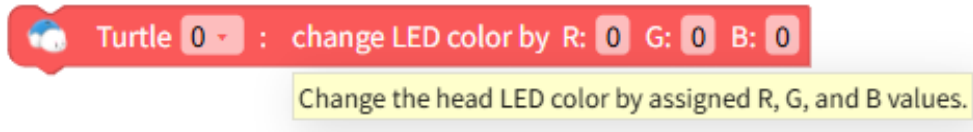
```
# Set head LED color to R:255, G:0, B:0
__('Turtle*0:head_led').d = [255, 0, 0]

# Set head LED color to R:255, G:255, B:0
__('Turtle*0:head_led').d = [255, 255, 0]

# Set head LED color to random color
__('Turtle*0:head_led').d = __randomColor()
```


Change LED Color by Specified RGB Amount

Changes the Turtle's head LED color by the specified R, G, B values.



Dropdown Options and Input Values

Name	Type	Description	Range / Type
color	Input Value	Delta R, G, B values	Integers for each of R, G, B: Integer -255 ~ 255

JavaScript Code

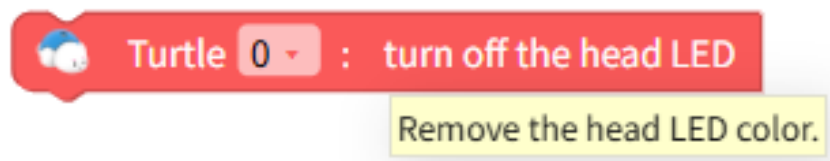
```
// Change head LED color by R:+1, G:+2, B:+3
$('Turtle*0:head_led').d = [
  $('Turtle*0:head_led').d[0] + 1,
  $('Turtle*0:head_led').d[1] + 2,
  $('Turtle*0:head_led').d[2] + 3];
```

Python Code

```
# Change head LED color by R:+1, G:+2, B:+3
__('Turtle*0:head_led').d = [
  __('Turtle*0:head_led').d[0] + 1,
  __('Turtle*0:head_led').d[1] + 2,
  __('Turtle*0:head_led').d[2] + 3]
```

Turn Off LED

Turns off the Turtle's head LED.



JavaScript Code

```
// Turn off head LED
$('Turtle*0:head_led').d = [0, 0, 0];
```

Python Code

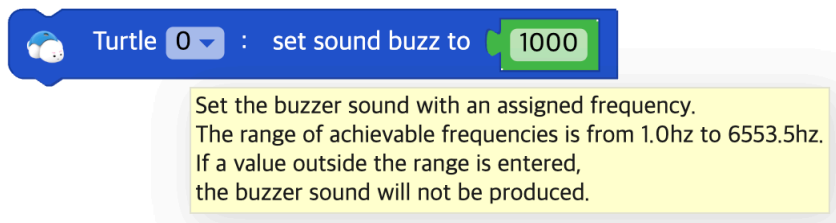
```
# Turn off head LED
__('Turtle*0:head_led').d = [0, 0, 0]
```

Set Buzzer Frequency

Sets the Turtle's buzzer sound to the specified frequency.

The range of achievable frequencies is from 1.0hz to 6553.5hz.

If a value outside the range is entered, the buzzer sound will not be produced.



Dropdown Options and Input Values

Name	Type	Description	Range / Type
sound	Input Value	Buzzer frequency	1.0 ~ 6553.5 (hz)

JavaScript Code

```
// Set buzzer frequency to 1000hz
$('Turtle*0:sound.buzz').d = 1000;
```

Python Code

```
# Set buzzer frequency to 1000hz
__('Turtle*0:sound.buzz').d = 1000
```

Play a Note

Plays the specified musical note.



Play a specific note.

Dropdown Options and Input Values

Name	Type	Description	Range / Type
note	Dropdown Option	Note	Do, Do#, Re, Re#, Mi, Fa, Fa#, So, So#, La, La#, Si
octave	Dropdown Option	Octave	1 ~ 7

JavaScript Code

```
// Play octave 1 Do
$('Turtle*0:sound.note').d = 4;

// Play octave 1 Re
$('Turtle*0:sound.note').d = 6;

// Play octave 2 Do
$('Turtle*0:sound.note').d = 16;

// Play octave 7 Si
$('Turtle*0:sound.note').d = 87;
```

Python Code

```
# Play octave 1 Do
__('Turtle*0:sound.note').d = 4

# Play octave 1 Re
__('Turtle*0:sound.note').d = 6

# Play octave 2 Do
__('Turtle*0:sound.note').d = 16

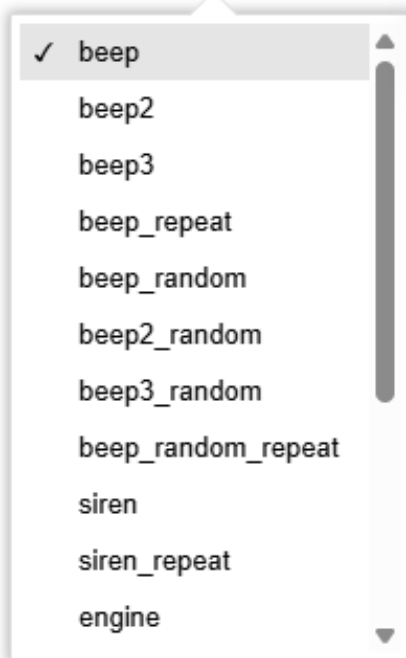
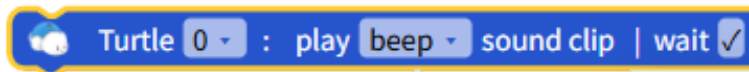
# Play octave 7 Si
__('Turtle*0:sound.note').d = 87
```

Play a Sound Clip

Plays a specific sound clip.

If you check “wait”, it waits until playback completes.

If you check “wait”, it can only be used inside an async function.



Play a specific sound clip.
If wait is checked, it will wait until playing sound clip completes.

Dropdown Options and Input Values

Name	Type	Description	Range / Type
sound_clip	Dropdown Option	Sound clip	beep(1), beep2(2), beep3(3), beep_repeat(4), beep_random(5), beep2_random(6), beep3_random(7), beep_random_repeat(8), siren(16), siren_repeat(17), engine(32), engine_repeat(33), noise_random(34), r2d2(48), toy_march(64), birthday(65), dibidibidip(66), mission_complete(67)

JavaScript Code

```
// Play dibidibidip(66) | Wait 0
$('Turtle*0:sound.clip').d = 66;
await $('Turtle*0:sound.!clip').w();

// Play mission_complete(67) | Wait X
$('Turtle*0:sound.clip').d = 67;
```

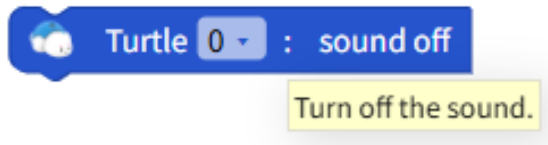
Python Code

```
# Play dibidibidip(66) | Wait 0
__('Turtle*0:sound.clip').d = 66
await __('Turtle*0:sound.!clip').w()
```

```
# Play mission_complete(67) | Wait X  
__('Turtle*0:sound.clip').d = 67
```

Stop Sound

Turns off the Turtle's sound.



JavaScript Code

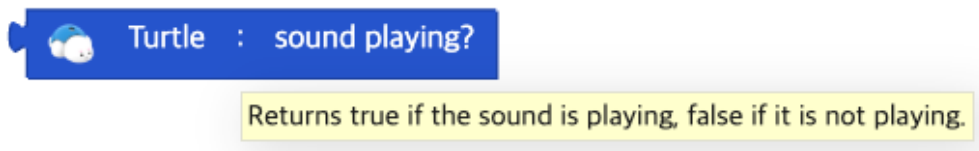
```
// Stop Turtle sound  
__stopSound('Turtle*0');
```

Python Code

```
# Stop Turtle sound  
__stopSound('Turtle*0')
```

Is Sound Playing?

Returns whether the Turtle's sound is playing as **True (1) / False (0)**.



JavaScript Code

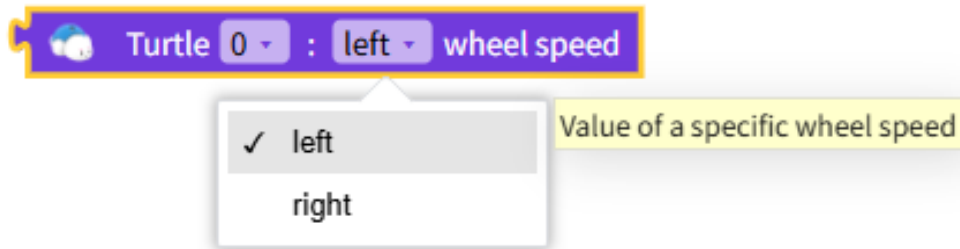
```
// Is Turtle sound playing? - true if playing, false otherwise  
$('Turtle*0:sound.playing').d;
```

Python Code

```
# Is Turtle sound playing? - True if playing, False otherwise
__('Turtle*0:sound.playing').d
```

Wheel Speed Value

Returns the specified wheel speed value.



Dropdown Options and Input Values

Name	Type	Description	Range / Type
direction	Dropdown Option	Direction	left, right

JavaScript Code

```
// Left wheel speed
__getSpeedInput('Turtle*0', $('Turtle*0:wheel.speed.left').d);

// Right wheel speed
__getSpeedInput('Turtle*0', $('Turtle*0:wheel.speed.right').d);
```

Python Code

```
# Left wheel speed
__getSpeedInput('Turtle*0', __('Turtle*0:wheel.speed.left').d)

# Right wheel speed
__getSpeedInput('Turtle*0', __('Turtle*0:wheel.speed.right').d)
```

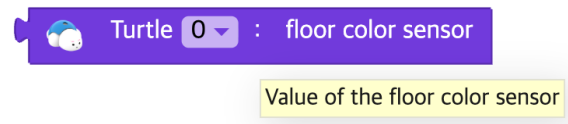
Floor Color Sensor Value

Returns the Turtle's floor color sensor value.

It recognizes black or colored lines drawn on the floor.

It is automatically adjusted based on A4 paper, 0 is black and 100 is white.

When a color is recognized, it approaches the black value.



JavaScript Code

```
// Floor Color sensor value  
$('Turtle*0:floor').d;
```

Python Code

```
# Floor Color sensor value  
__('Turtle*0:floor').d
```

Card Color Number Value

Returns the card color number read by the bottom color sensor.

Color	Value
Black	0
Red	1
Orange	2
Yellow	3
Green	4
Cyan	5
Blue	6
Magenta	7

Color	Value
White	8

 Turtle 0 : card color number

Number of card color read through the color sensor

JavaScript Code

```
// Read card color number value
$('Turtle*0:color.name').d;
```

Python Code

```
# Read card color number value
__('Turtle*0:color.name').d
```

Card Color Pattern Value

Returns the 'color pattern' of the card read by the color sensor under the Turtle's head.

- * How to recognize: Pass the card through the sensor in the order of white strip -> color strip -> background.
- * Reading Result: The result appears as a two-digit number, representing the [tens digit = color strip] and [units digit = background color].
- * Example: When a card with 'purple(7) strip' and 'red(1) background' is read, the result is 71.

 Turtle 0 : card color pattern

Pattern of card color read through the color sensor

JavaScript Code

```
// Card color pattern value
$('Turtle*0:card').d;
```

Python Code

```
# Card color pattern value
__('Turtle*0:card').d
```

Acceleration Value

Returns the acceleration value for a specific axis.

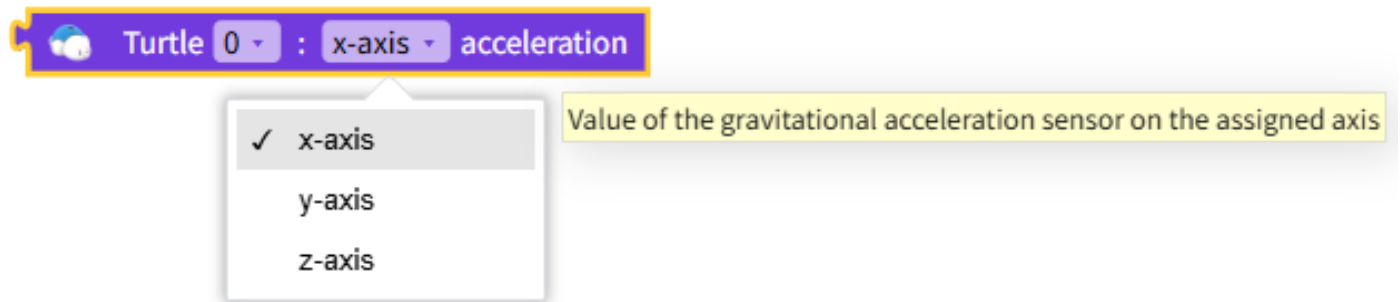
The accelerometer is a sensor that measures how quickly the Turtle moves or tilts.

The range is -128 to +127.

* X-axis: Changes when tilting the robot forward or backward.

* Y-axis: Changes when tilting the robot left or right.

* Z-axis: Changes when flipping the robot up or down.



Dropdown Options and Input Values

Name	Type	Description	Range / Type
axis	Dropdown Option	Axis	x, y, z

JavaScript Code

```
// Acceleration value on x-axis
$('Turtle*0:acceleration.x').d;

// Acceleration value on y-axis
$('Turtle*0:acceleration.y').d;

// Acceleration value on z-axis
```

```
$('Turtle*0:acceleration.z').d;
```

Python Code

```
# Acceleration value on x-axis
__('Turtle*0:acceleration.x').d

# Acceleration value on y-axis
__('Turtle*0:acceleration.y').d

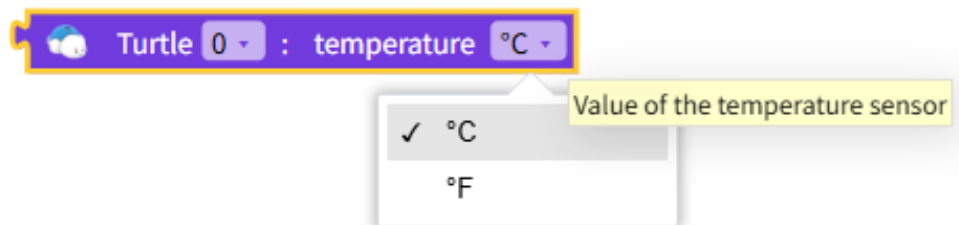
# Acceleration value on z-axis
__('Turtle*0:acceleration.z').d
```

Temperature Sensor Value

Returns the Turtle's temperature sensor value.

It represents the internal temperature of the robot, not the room temperature.

The range is -40.0 to +87.5 degrees(°C).



Dropdown Options and Input Values

Name	Type	Description	Range / Type
unit	Dropdown Option	Temperature unit	°C, °F

JavaScript Code

```
// Temperature in Celsius
__getTemperature($('Turtle*0:temperature').d, '°C');
```

```
// Temperature in Fahrenheit
__getTemperature($('Turtle*0:temperature').d, '°F');
```

Python Code

```
# Temperature in Celsius
__getTemperature(__('Turtle*0:temperature').d, '°C')

# Temperature in Fahrenheit
__getTemperature(__('Turtle*0:temperature').d, '°F')
```

Signal Strength Value

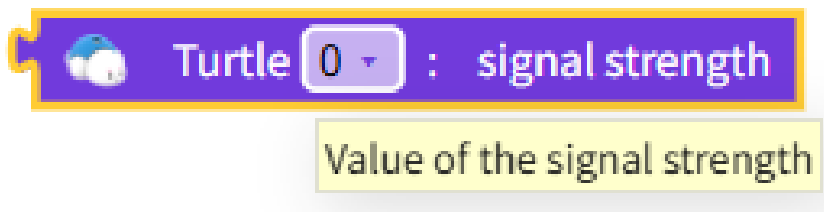
Returns the Turtle's signal strength value.

It indicates the signal strength between the robot and the dongle.

The value increases as the distance decreases.

For the stable operation, it is recommended to use within the range of -50dBm ~ -80dBm.

If the signal strength value is closer to 0, the signal is stronger. (e.g. -40 > -90)



JavaScript Code

```
// Signal strength value
$('Turtle*0:signal_strength').d;
```

Python Code

```
# Signal strength value
__('Turtle*0:signal_strength').d
```

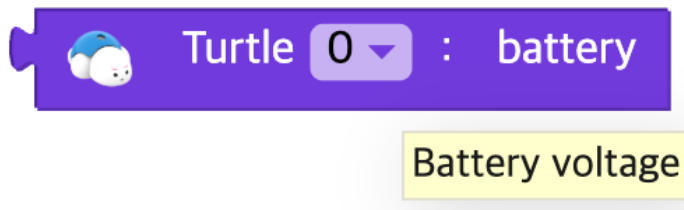
Battery Voltage

Returns the Turtle's battery voltage value.

The voltage is approximatedly 4.0V when the battery is fully charged.

It typically operates stably at 3.7V or higher.

Since operating errors may occur if the voltage drops below 3.6V, charging is strongly recommended.



JavaScript Code

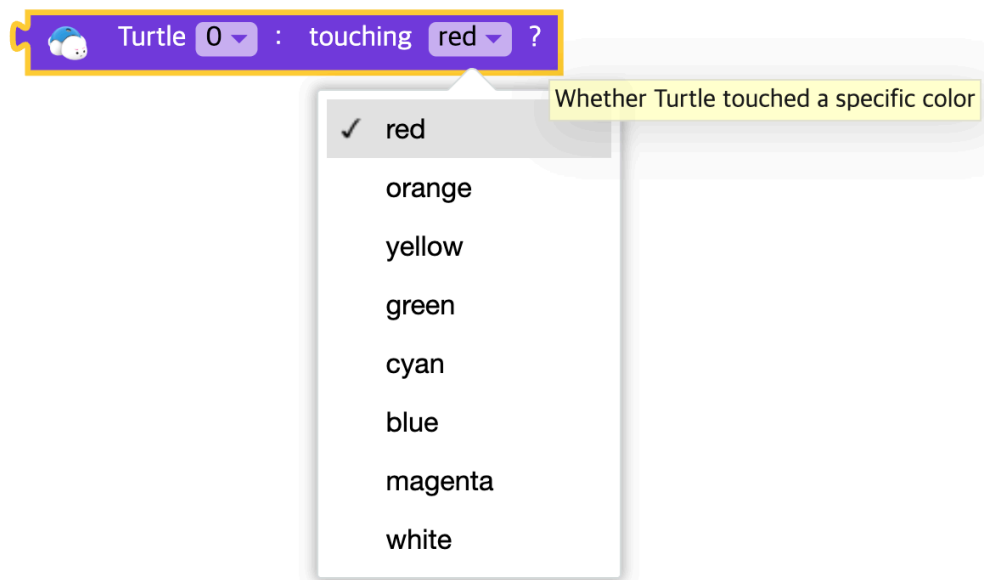
```
// Battery voltage value
$('Turtle*0:battery').d;
```

Python Code

```
# Battery voltage value
__('Turtle*0:battery').d
```

Is Touching a Specific Color?

Using the Turtle's color sensor, returns whether it is touching the specified color as **True (1) / False (0)**.



Dropdown Options and Input Values

Name	Type	Description	Range / Type
color	Dropdown Option	Detected color	red(1), orange(2), yellow(3), green(4), cyan(5), blue(6), magenta(7), white(8)

JavaScript Code

```
// Touching red?
$('Turtle*0:color.name').d === 1;

// Touching orange?
$('Turtle*0:color.name').d === 2;

// Touching yellow?
$('Turtle*0:color.name').d === 3;

// Touching green?
$('Turtle*0:color.name').d === 4;
```

```
// Touching cyan?
$('Turtle*0:color.name').d === 5;

// Touching blue?
$('Turtle*0:color.name').d === 6;

// Touching magenta?
$('Turtle*0:color.name').d === 7;

// Touching white?
$('Turtle*0:color.name').d === 8;
```

Python Code

```
# Touching red?
__('Turtle*0:color.name').d == 1

# Touching orange?
__('Turtle*0:color.name').d == 2

# Touching yellow?
__('Turtle*0:color.name').d == 3

# Touching green?
__('Turtle*0:color.name').d == 4

# Touching cyan?
__('Turtle*0:color.name').d == 5

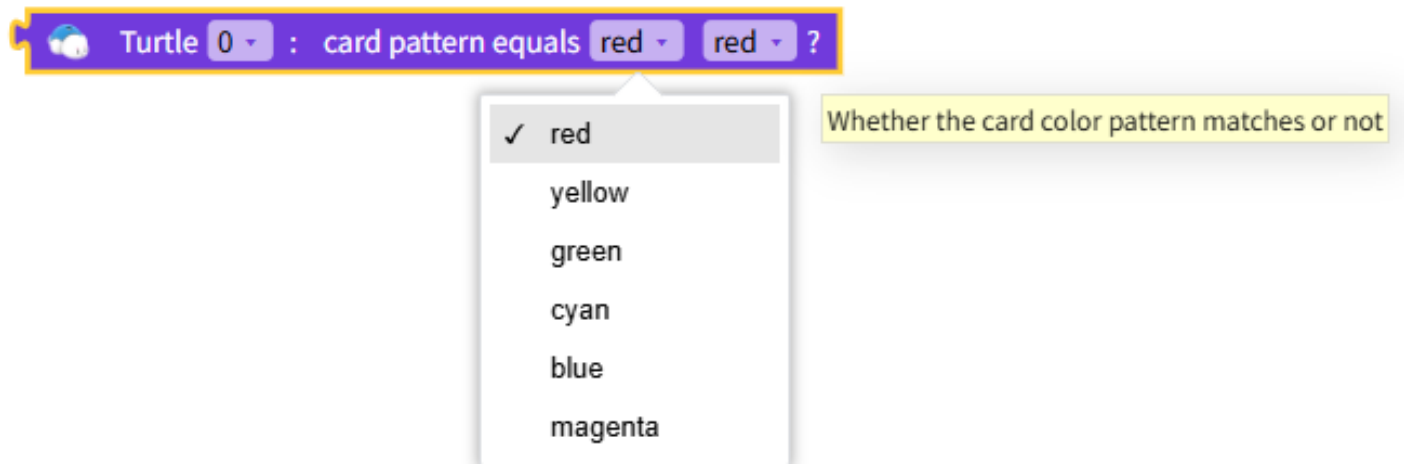
# Touching blue?
__('Turtle*0:color.name').d == 6

# Touching magenta?
__('Turtle*0:color.name').d == 7
```

```
# Touching white?
__('Turtle*0:color.name').d == 8
```

Is the Card Color Pattern ~ ?

Depending on whether the detected card color pattern matches, returns **True (1) / False (0)**.



Dropdown Options and Input Values

Name	Type	Description	Range / Type
pattern 1	Dropdown Option	Card color pattern	red, yellow, green, cyan, blue, magenta
pattern 2	Dropdown Option	Card color pattern	red, yellow, green, cyan, blue, magenta

JavaScript Code

```
// Is card pattern red-red?
$('Turtle*0:card').d === 9;

// Is card pattern red-yellow?
$('Turtle*0:card').d === 10;
```



```

// Is card pattern red-green?
$('Turtle*0:card').d === 11;

// Is card pattern red-cyan?
$('Turtle*0:card').d === 12;

// Is card pattern red-blue?
$('Turtle*0:card').d === 13;

// Is card pattern red-magenta?
$('Turtle*0:card').d === 14;

// Is card pattern yellow-red?
$('Turtle*0:card').d === 17;

// Is card pattern green-red?
$('Turtle*0:card').d === 25;

// Is card pattern cyan-red?
$('Turtle*0:card').d === 33;

// Is card pattern blue-red?
$('Turtle*0:card').d === 41;

// Is card pattern magenta-red?
$('Turtle*0:card').d === 49;

// Is card pattern magenta-magenta?
$('Turtle*0:card').d === 54;

```

Python Code

```

# Is card pattern red-red?
__('Turtle*0:card').d == 9

```

```
# Is card pattern red-yellow?
__('Turtle*0:card').d == 10

# Is card pattern red-green?
__('Turtle*0:card').d == 11

# Is card pattern red-cyan?
__('Turtle*0:card').d == 12

# Is card pattern red-blue?
__('Turtle*0:card').d == 13

# Is card pattern red-magenta?
__('Turtle*0:card').d == 14

# Is card pattern yellow-red?
__('Turtle*0:card').d == 17

# Is card pattern green-red?
__('Turtle*0:card').d == 25

# Is card pattern cyan-red?
__('Turtle*0:card').d == 33

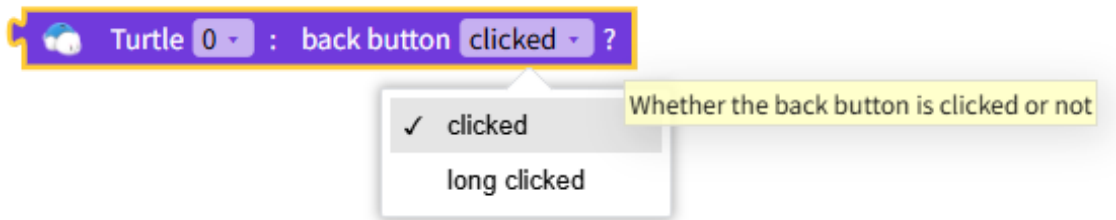
# Is card pattern blue-red?
__('Turtle*0:card').d == 41

# Is card pattern magenta-red?
__('Turtle*0:card').d == 49

# Is card pattern magenta-magenta?
__('Turtle*0:card').d == 54
```

Was the Back Button Clicked?

Depending on back button click status, returns **True (1) / False (0)**.



Dropdown Options and Input Values

Name	Type	Description	Range / Type
action	Dropdown Option	Click type	click, long_click

JavaScript Code

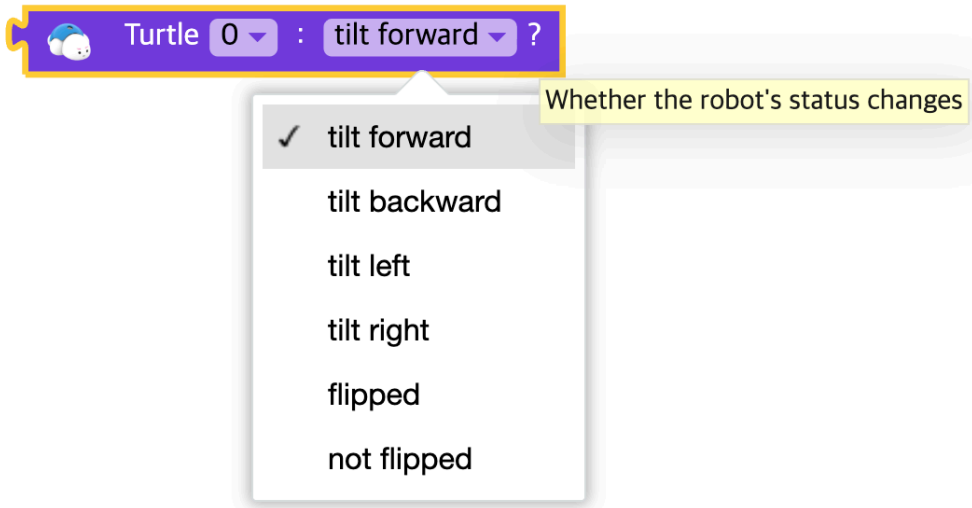
```
// Was back button clicked?  
$('Turtle*0:button.click').e;  
  
// Was back button long-clicked?  
$('Turtle*0:button.long_click').e;
```

Python Code

```
# Was back button clicked?  
__('Turtle*0:button.click').e  
  
# Was back button long-clicked?  
__('Turtle*0:button.long_click').e
```

State Change Condition

Depending on the condition, returns whether the Turtle's state has changed as **True (1) / False (0)**.



Dropdown Options and Input Values

Name	Type	Description	Range / Type
condition	Dropdown Option	Pose condition	tilted forward, tilted backward, tilted left, tilted right, flipped over, not flipped

JavaScript Code

```
// Is Turtle tilted forward?
$('Turtle*0:acceleration.x').d > 50;

// Is Turtle tilted backward?
$('Turtle*0:acceleration.x').d < -50;

// Is Turtle tilted left?
$('Turtle*0:acceleration.y').d > 50;

// Is Turtle tilted right?
$('Turtle*0:acceleration.y').d < -50;

// Is Turtle flipped over?
```

```
$('Turtle*0:acceleration.z').d > 0;  
  
// Is Turtle not flipped?  
$('Turtle*0:acceleration.z').d < -30;
```

Python Code

```
# Is Turtle tilted forward?  
__('Turtle*0:acceleration.x').d > 50  
  
# Is Turtle tilted backward?  
__('Turtle*0:acceleration.x').d < -50  
  
# Is Turtle tilted left?  
__('Turtle*0:acceleration.y').d > 50  
  
# Is Turtle tilted right?  
__('Turtle*0:acceleration.y').d < -50  
  
# Is Turtle flipped over?  
__('Turtle*0:acceleration.z').d > 0  
  
# Is Turtle not flipped?  
__('Turtle*0:acceleration.z').d < -30
```